

UNIVERSIDADE DE FORTALEZA

Fundação Edson Queiroz

Curso de Engenharia da Computação

RELATÓRIO DE ATIVIDADE PRÁTICA: AV1

**Comunicação Serial via UART1 com Leitura Analógica (ADC)
na Placa NUCLEO-STM32F446RE**

Disciplina: Sistema de Tempo Real (STR)

Professor: Imbiriba

Aluno: Rafael Marins

Fortaleza, Setembro de 2026

1. Introdução

Sistemas embarcados frequentemente precisam trocar informações com o mundo externo, seja para monitorar grandezas físicas por meio de sensores, seja para reportar seu estado a um operador humano ou a outro sistema. Duas interfaces fundamentais para isso em microcontroladores da família STM32 são a comunicação serial assíncrona (USART/UART) e o conversor analógico-digital (ADC).

Este relatório documenta o desenvolvimento de uma atividade prática que integra esses dois periféricos na placa NUCLEO-STM32F446RE, programada em linguagem C com acesso direto aos registradores dos periféricos (sem uso da biblioteca HAL), seguindo a mesma metodologia adotada nas Práticas 1 a 4 da disciplina. O ponto de partida foi o código da Prática 3 (Comunicação Serial), Parte 5: que implementa recepção serial por interrupção com toggle de um LED, adaptado para operar através da USART1 (em vez da USART2 utilizada originalmente) e para controlar o pino GPIOC10. A esse código foi somada a leitura do canal 1 do conversor analógico-digital (ADC1), desenvolvida originalmente na Prática 4, convertendo a leitura de 12 bits em um valor de tensão (em Volts) e transmitindo-o continuamente pela mesma interface serial.

Além da implementação em si, este relatório também documenta o processo de depuração de um problema real de configuração do ambiente de desenvolvimento, identificado durante os testes em bancada, cuja resolução exigiu investigação sistemática dos registradores do periférico e da configuração de ponto flutuante do compilador, um exercício representativo dos desafios comuns no desenvolvimento de firmware bare-metal.

2. Objetivos

Objetivo geral: adaptar e integrar os drivers de comunicação serial (USART) e de conversão analógico-digital (ADC) desenvolvidos nas Práticas 3 e 4, implementando na placa NUCLEO-STM32F446RE um sistema que recebe caracteres pela USART1 com toggle de um pino de saída a cada byte recebido, via interrupção e que transmite continuamente, pela mesma interface, a leitura de tensão do canal 1 do ADC1.

Objetivos específicos:

- Configurar a interface USART1 (pinos PA9/TX e PA10/RX) por acesso direto a registradores, incluindo geração do baud rate, habilitação de transmissão/recepção e configuração da interrupção de recepção (RXNEIE) no controlador de interrupções (NVIC);
- Redirecionar a função de biblioteca printf() para a USART1, através da implementação da função de baixo nível __io_putchar();
- Implementar o toggle do pino GPIOC10 a cada byte recebido pela USART1, dentro da rotina de tratamento de interrupção (USART1_IRQHandler);
- Configurar o ADC1 para realizar conversões do canal 1 (pino PA1) em modo de conversão única, com verificação do fim de conversão por meio da flag EOC (End Of Conversion);
- Converter o valor digital de 12 bits (0 a 4095) lido do ADC para um valor de tensão em Volts, considerando a referência de 3,3 V da própria placa;

- Validar o funcionamento do sistema na placa física, incluindo a depuração de falhas encontradas durante os testes em bancada.

3. Revisão Bibliográfica

A comunicação USART (Universal Synchronous/Asynchronous Receiver/Transmitter) é um periférico amplamente utilizado em microcontroladores para troca de dados seriais, podendo operar tanto no modo síncrono quanto no assíncrono. No modo assíncrono (UART), a transmissão ocorre sem um sinal de clock compartilhado entre os dispositivos: a sincronização é obtida por meio de bits de início (start bit) e de parada (stop bit) que delimitam cada byte transmitido, a uma taxa combinada previamente entre os dispositivos (baud rate) [1]. Nos microcontroladores da família STM32F4, cada interface USART possui um conjunto de registradores mapeados em memória, como o registrador de controle (USART_CR1), o registrador de status (USART_SR), o registrador de dados (USART_DR) e o registrador de configuração do baud rate (USART_BRR) que permitem a configuração e a operação do periférico por acesso direto a esses endereços, sem a necessidade de bibliotecas intermediárias [1].

O conversor analógico-digital (ADC) presente nos microcontroladores STM32F4 utiliza a técnica de aproximação sucessiva, na qual a tensão de entrada é comparada sucessivamente a valores de referência gerados internamente, refinando a estimativa a cada ciclo até atingir a resolução configurada, até 12 bits, ou 4096 níveis, nos dispositivos utilizados [1][5]. O resultado da conversão fica disponível no registrador de dados do ADC (ADC_DR), podendo ser lido tanto por varredura (polling) da flag de fim de conversão (EOC) quanto por interrupção [5].

Ambos os periféricos utilizam a arquitetura de barramentos do STM32F4, na qual cada periférico é conectado a um barramento específico (AHB1, APB1 ou APB2) e deve ter seu clock individualmente habilitado através dos registradores do módulo de Reset and Clock Control (RCC) antes de ser utilizado [1]. Além disso, os pinos de propósito geral (GPIO) que servem de interface física para USART e ADC precisam ser configurados quanto ao seu modo de operação (entrada, saída, função alternativa ou analógica) e, no caso de periféricos com função alternativa, à função específica associada por meio dos registradores GPIOx_AFRL/AFRH [1].

Por fim, no núcleo Arm Cortex-M4 presente no STM32F446RE, operações de ponto flutuante executadas em hardware dependem do coprocessador de FPU (Floating Point Unit), cujo acesso deve ser explicitamente habilitado por software, através do registrador CPACR (Coprocessor Access Control Register), antes de qualquer instrução de ponto flutuante ser executada; essa habilitação é tipicamente realizada pela função `SystemInit()`, fornecida pelos arquivos de inicialização (CMSIS) da ST [2][3]. Essa característica arquitetural teve papel central na depuração relatada na Seção 5 deste documento.

4. Metodologia

4.1 Ambiente de desenvolvimento

O desenvolvimento foi realizado no STM32CubeIDE 2.2.0, em um projeto configurado para o microcontrolador STM32F446RETx (placa física NUCLEO-F446RE), utilizando a mesma abordagem bare-

metal (acesso direto a registradores, sem HAL) empregada nas Práticas 1 a 4. O código foi organizado de forma modular, em drivers separados por periférico (uart.c/uart.h e adc.c/adc.h) e um arquivo main.c responsável pela orquestração geral, seguindo o mesmo padrão adotado nas apostilas da disciplina.

Observa-se que, embora o Manual de Referência (RM0383) disponível para consulta trate especificamente da família STM32F411, os registradores de GPIO, RCC, USART e ADC utilizados neste trabalho são idênticos entre as variantes STM32F401, STM32F411 e STM32F446, por pertencerem ao mesmo bloco de propriedade intelectual (IP) compartilhado dentro da família STM32F4. Assim, a consulta ao RM0383 é válida como referência de registradores mesmo para a placa STM32F446RE efetivamente utilizada.

4.2 Ponto de partida: Prática 3, Parte 5

A Parte 5 da Prática 3 implementa recepção serial via interrupção na USART2 (pinos PA2/TX e PA3/RX): o bit RXNEIE do registrador USART_CR1 é habilitado, assim como a interrupção correspondente no NVIC; a cada byte recebido, a rotina USART2_IRQHandler invoca uma função de callback que lê o dado recebido (USART2->DR), realiza o toggle de um pino de saída (PA5, ligado ao LED de usuário da placa) e imprime o caractere recebido via printf().

4.3 Adaptação para USART1 e GPIOC10

A Tabela 1 resume as diferenças entre a configuração original (USART2) e a configuração adotada neste trabalho (USART1), mantendo a mesma lógica de recepção por interrupção.

Aspecto	Prática 3 – Parte 5 (original)	Este trabalho
Interface USART	USART2 (barramento APB1)	USART1 (barramento APB2)
Pino TX	PA2 (AF7)	PA9 (AF7)
Pino RX	PA3 (AF7)	PA10 (AF7)
Registrador de função alternativa	GPIOA_AFRL (AFR[0])	GPIOA_AFRH (AFR[1])
Bit de habilitação de clock	RCC_APB1ENR, bit 17 (USART2EN)	RCC_APB2ENR, bit 4 (USART1EN)
Pino de LED (toggle no RX)	PA5	PC10
Baud rate	9600 bps	9600 bps

A principal particularidade identificada nessa adaptação foi que os pinos PA9 e PA10 pertencem à faixa de pinos 8 a 15 de cada porta GPIO, cuja função alternativa é selecionada pelo registrador GPIOx_AFRH diferente dos pinos PA2/PA3 (faixa 0 a 7), configurados pelo GPIOx_AFRL. Além disso, a USART1 está conectada ao barramento APB2, exigindo a habilitação do seu clock através do registrador RCC_APB2ENR (bit USART1EN), em vez do RCC_APB1ENR utilizado para a USART2.

O pino GPIOC10, solicitado pela atividade para o toggle do LED, não possui nenhum LED de fábrica conectado a ele em nenhuma placa da família NUCLEO-64 (esse recurso está fixo ao pino PA5, LED de usuário LD2, conforme o manual da placa [4]). A correta execução do toggle foi confirmada por meio da impressão do caractere recebido a cada interrupção (ver Seção 5), já que essa impressão ocorre na mesma rotina de callback que realiza o toggle do pino.

4.4 Leitura do ADC1 (canal 1) e conversão para Volts

A leitura analógica foi implementada com base na Prática 4 (Tarefa 2), configurando o ADC1 para realizar conversões de canal único no pino PA1 (canal 1), da seguinte forma:

- Configuração do pino PA1 em modo analógico (GPIOA_MODER = 11) e habilitação do clock do ADC1 (RCC_APB2ENR, bit ADC1EN);
- Seleção do canal 1 como única conversão da sequência regular (ADC1_SQR3 = 1) e configuração do comprimento da sequência para uma conversão (ADC1_SQR1 = 0);
- Habilitação do módulo (ADC1_CR2, bit ADON) e disparo de cada conversão pelo bit SWSTART do mesmo registrador;
- Espera ativa (polling) da flag EOC (End Of Conversion) no registrador ADC1_SR antes da leitura do resultado em ADC1_DR.

O valor bruto de 12 bits (0 a 4095) foi convertido para tensão considerando a referência de 3,3 V da própria placa, segundo a Equação 1:

$$V = (\text{valor_bruto} \times 3,3) / 4095$$

O valor calculado é transmitido continuamente pela USART1 através da função printf(), redirecionada para a USART1 por meio da implementação de __io_putchar(). Como o cálculo e a formatação envolvem números de ponto flutuante, foi necessário habilitar, nas configurações do projeto (MCU/MPU Settings), a opção de vinculação da biblioteca de formatação de float do newlib-nano (-u _printf_float).

4.5 Validação física

Para a validação em bancada, a USART1 foi conectada a um computador através de um adaptador USB-Serial (USB-TTL) externo, já que a interface ST-LINK da placa está cabeada, por padrão de fábrica, à USART2, não à USART1 utilizada neste trabalho. A ligação foi feita de forma cruzada (TXD do adaptador ao pino PA10/RX, e RXD do adaptador ao pino PA9/TX), com o GND do adaptador ligado a um pino GND da placa, sem uso de fio de alimentação entre os dois dispositivos, já que a Nucleo possui sua própria fonte via ST-LINK. A visualização dos dados no computador (sistema operacional Linux) foi feita através do terminal serial minicom, configurado para 9600 bps.

5. Resultados e Discussão

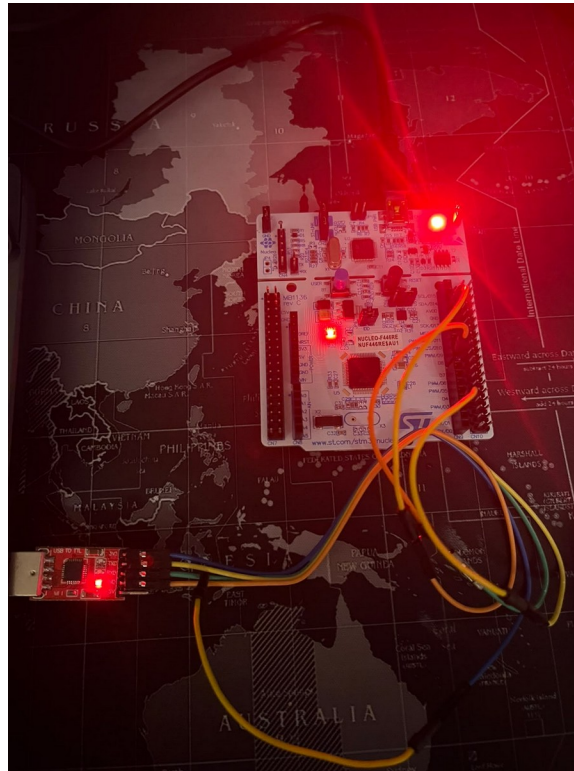


Figura 1 — Montagem física utilizada nos testes: placa NUCLEO-STM32F446RE conectada a um adaptador USB-TTL (pinos PA9/PA10/GND).

5.1 Depuração de um problema real de ambiente: FPU não habilitada

Durante os primeiros testes na placa física, observou-se um comportamento peculiar: a mensagem de inicialização era transmitida corretamente pela USART1, porém nenhuma leitura de tensão era impressa em seguida, o sistema parecia travar silenciosamente. A investigação foi conduzida de forma sistemática, por meio de impressões de diagnóstico inseridas em pontos específicos do código (já que a via de depuração por breakpoints não estava disponível na IDE no momento do teste):

- Impressão dos valores dos registradores RCC_APB2ENR, ADC1_CR2 e ADC1_SQR3 logo após a inicialização do ADC, confirmando que a configuração estava correta;
- Impressões antes e depois da espera pela flag EOC dentro da rotina de leitura do ADC, confirmando que a conversão era concluída com sucesso;
- Constatação de que o travamento ocorria exatamente na primeira operação aritmética de ponto flutuante do programa (o cálculo da tensão em Volts), mesmo quando a formatação com `printf("%f")` era temporariamente substituída por uma mensagem de texto simples, evidenciando que a causa não estava na formatação, mas na própria instrução de ponto flutuante.

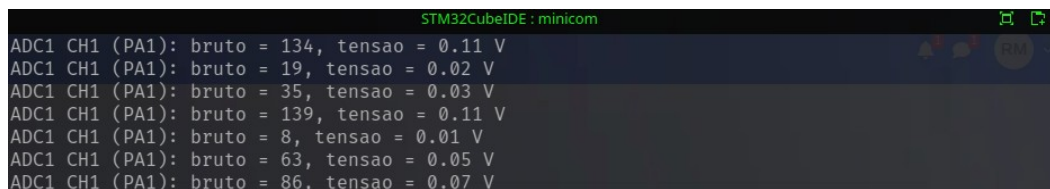
A causa raiz identificada foi a ausência do arquivo `system_stm32f4xx.c` na pasta de código-fonte do projeto. Esse arquivo contém a implementação real da função `SystemInit()`, responsável por habilitar o acesso ao coprocessador de FPU através do registrador CPACR [2][3]. Como o projeto foi montado copiando manualmente as pastas de drivers (sem uso do assistente gráfico da IDE), esse arquivo

específico localizado dentro de uma subpasta de "templates" não havia sido incluído. Na ausência dele, o arquivo de inicialização (startup_stm32f446retx.s) utiliza uma implementação vazia (fraca) de SystemInit(), fornecida apenas para que o processo de linkedição não falhe. Como o projeto está configurado para usar a FPU em hardware (Floating-point ABI = hard), a primeira instrução de ponto flutuante executada sem o coprocessador habilitado provoca uma falha de hardware, que é tratada pelo manipulador padrão de falhas (HardFault_Handler), implementado como um laço infinito.

A correção consistiu em copiar o arquivo system_stm32f4xx.c da pasta de templates dos drivers (Drivers/CMSIS/Device/ST/STM32F4xx/Source/Templates/) para a pasta Src/ do projeto. Após a correção, o sistema passou a executar continuamente, sem travamentos, conforme evidenciado nas figuras a seguir. Esse episódio ilustra um problema comum em projetos bare-metal montados manualmente, a omissão de um arquivo de inicialização essencial, cuja ausência não impede a compilação nem a linkedição, apenas se manifesta em tempo de execução.

5.2 Validação da leitura do ADC

Com o pino PA1 desconectado (flutuando), as leituras apresentaram valores baixos e instáveis, como esperado para uma entrada analógica sem sinal definido, o pino capta ruído eletromagnético do ambiente, resultando em conversões que variam a cada amostra (Figura 2).

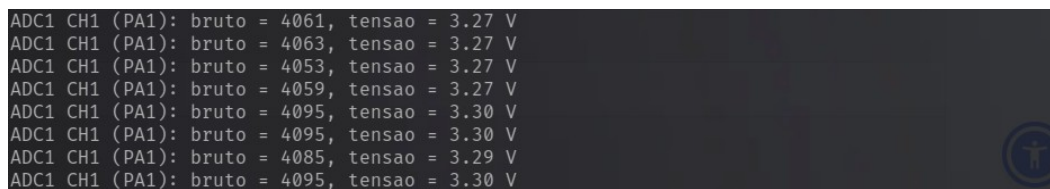


STM32CubeIDE : minicom

```
ADC1 CH1 (PA1): bruto = 134, tensao = 0.11 V
ADC1 CH1 (PA1): bruto = 19, tensao = 0.02 V
ADC1 CH1 (PA1): bruto = 35, tensao = 0.03 V
ADC1 CH1 (PA1): bruto = 139, tensao = 0.11 V
ADC1 CH1 (PA1): bruto = 8, tensao = 0.01 V
ADC1 CH1 (PA1): bruto = 63, tensao = 0.05 V
ADC1 CH1 (PA1): bruto = 86, tensao = 0.07 V
```

Figura 2 — Leituras do ADC1 com o pino PA1 desconectado (entrada flutuante): valores baixos e instáveis, causados por ruído captado pelo pino.

Em seguida, o pino PA1 foi conectado diretamente ao pino de 3,3 V da própria placa, como forma de calibração/validação da escala de conversão. Como esperado, as leituras se estabilizaram próximas do valor máximo da escala (Figura 3), com o valor bruto atingindo o máximo teórico de 4095 em parte das amostras e a tensão calculada variando entre 3,27 V e 3,30 V um erro inferior a 1% em relação ao valor de referência esperado (3,3 V), compatível com a resolução de 12 bits do conversor (passo de aproximadamente 0,8 mV) e com pequenas variações da própria fonte de alimentação da placa.



```
ADC1 CH1 (PA1): bruto = 4061, tensao = 3.27 V
ADC1 CH1 (PA1): bruto = 4063, tensao = 3.27 V
ADC1 CH1 (PA1): bruto = 4053, tensao = 3.27 V
ADC1 CH1 (PA1): bruto = 4059, tensao = 3.27 V
ADC1 CH1 (PA1): bruto = 4095, tensao = 3.30 V
ADC1 CH1 (PA1): bruto = 4095, tensao = 3.30 V
ADC1 CH1 (PA1): bruto = 4085, tensao = 3.29 V
ADC1 CH1 (PA1): bruto = 4095, tensao = 3.30 V
```

Figura 3 — Leituras do ADC1 com o pino PA1 conectado diretamente a 3,3 V: valores estáveis, próximos do fundo de escala, confirmando o correto funcionamento da conversão e da equação de escala utilizada.

5.3 Validação da recepção por interrupção

O funcionamento da recepção por interrupção e do toggle do pino GPIOC10 foi validado indiretamente: como o pino GPIOC10 não possui LED de fábrica conectado em nenhuma placa da família NUCLEO-64

(característica do desenho da placa, e não do microcontrolador específico utilizado), optou-se por habilitar temporariamente, dentro da rotina de callback de recepção, a impressão do caractere recebido pela USART1. Ao digitar caracteres no terminal serial, cada um deles foi corretamente ecoado de volta pelo dispositivo, confirmando que a interrupção USART1_IRQHandler é executada a cada byte recebido e que, conseqüentemente, a instrução de toggle do pino GPIOC10, presente na mesma rotina, também é executada.

6. Conclusão

A atividade permitiu integrar, com sucesso, os drivers de comunicação serial e de conversão analógico-digital desenvolvidos ao longo da disciplina, adaptando-os para uma nova interface (USART1) e um novo pino de saída (GPIOC10), além de acrescentar a leitura contínua de uma grandeza analógica convertida para uma unidade de engenharia (Volts). Os testes em bancada confirmaram o funcionamento correto de todos os requisitos da atividade: recepção serial por interrupção com toggle do pino de saída, e leitura/transmissão contínua da tensão do canal 1 do ADC.

Mais relevante do que a implementação em si, o processo de depuração do problema da FPU não habilitada reforçou a importância de uma investigação sistemática e baseada em evidências ao lidar com falhas em sistemas embarcados bare-metal: a bisseção do problema por meio de impressões de diagnóstico, combinada à leitura direta dos registradores envolvidos, permitiu isolar a causa raiz com precisão, mesmo sem acesso imediato às ferramentas de depuração da IDE.

Como trabalho futuro, propõe-se a comparação da abordagem por polling (utilizada na leitura do ADC) com uma implementação por interrupção (ADC_IRQHandler), reduzindo o tempo de espera ativa do processador, conforme demonstrado na Parte 3 da Prática 4.

Referências

- [1] STMICROELECTRONICS. RM0383 — Reference manual: STM32F411xC/E advanced Arm®-based 32-bit MCUs. Rev. 3, nov. 2018.
- [2] STMICROELECTRONICS. PM0214 — STM32F3 Series, STM32F4 Series, STM32L4 Series and STM32L4+ Series Cortex®-M4 programming manual (referenciado em RM0383 para detalhes do núcleo Cortex-M4 com FPU).
- [3] STMICROELECTRONICS. Datasheet: STM32F411xC/STM32F411xE — Arm® Cortex®-M4 32b MCU+FPU. DocID026289 Rev 7, dez. 2017.
- [4] STMICROELECTRONICS. UM1724 — User manual: STM32 Nucleo-64 boards (MB1136). DocID025833 Rev 5, jun. 2014.
- [5] IMBIRIBA. Materiais didáticos das Práticas 1 a 4 — Disciplina Sistema de Tempo Real (STR). Universidade de Fortaleza (UNIFOR), 2026.